

Otimização de Hiperparâmetros



Turing USP

Gustavo Gomes Esquetini Barbosa
gustavo.esquetini@gmail.com

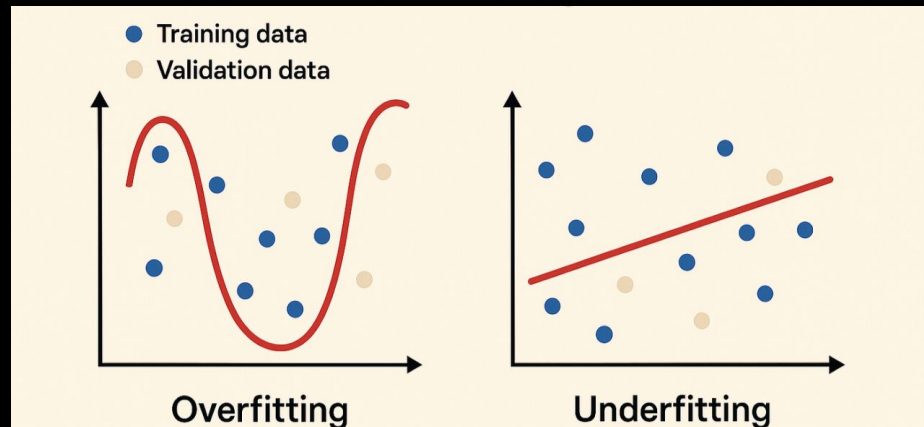
Recapitulando

Overfitting

Quando o modelo é mais simples do que deveria ser, ele não entende os dados e erra muito

Underfitting

Se o modelo é muito complexo ele começa a decorar e falha na hora da validação

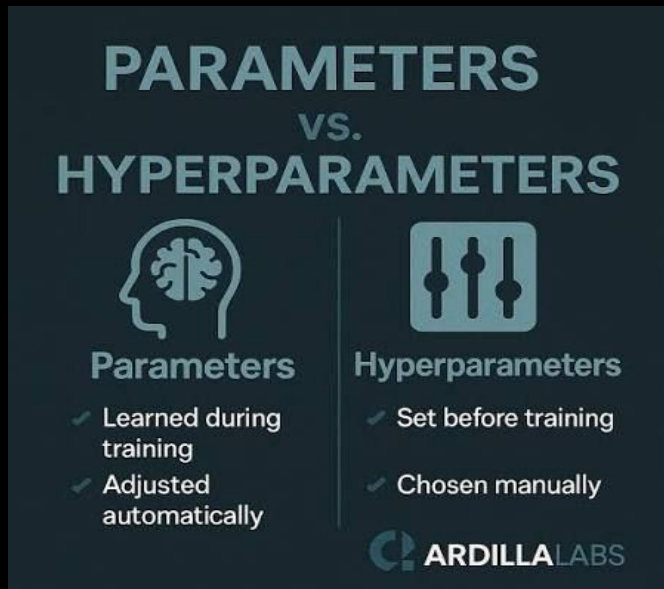


Como resolver este problema?

Guiando o Modelo

Aqui entram os Hiperparâmetros, eles são predefinições colocadas antes do modelo começar a treinar, eles incluem:

- a profundidade da árvore;
- número de estimadores;
- taxa de aprendizado (boosting);
- força da regularização;
- número de vizinhos (KNN);
- entre outros assuntos futuros



Como aplicar

Grid search

O método da força bruta, você define uma grade de valores possíveis e o algoritmo testa **TODAS** as combinações possíveis entre eles e o desempenho do modelo é avaliado usando alguma métrica específica.

- **Vantagem:** simplicidade e bom resultado
- **Desvantagem:** Inviável em projetos grandes

para os exemplos será utilizado o KNN.

Exemplo

Clássico:

```
model = KNeighborsClassifier()  
model.fit(X_train, y_train)
```

Grid search:

```
# hiperparâmetros do KNN a testar: 4 valores de n_neighbors x 2 tipos de weights = 8 combinações  
grid_search = {  
    'n_neighbors': [3, 5, 7, 9],          # hiperparâmetro do modelo  
    'weights': ['uniform', 'distance'] # hiperparâmetro do modelo  
}  
  
# cv=5 -> validação cruzada, não é hiperparâmetro do modelo  
# scoring -> métrica de avaliação: 'accuracy', 'precision', 'recall', 'f1', 'roc_auc', 'r2', 'neg_mean_squared_error'...  
grid = GridSearchCV(KNeighborsClassifier(), grid_search, cv=5, scoring='accuracy')  
  
# roda as 8 combinações x 5 folds = 40 treinos  
grid.fit(X_train, y_train)  
  
# melhor combinação encontrada  
print(grid.best_params_)
```

Como aplicar

Random search

Semelhante ao Grid search ele tem a mesma ideia, porém testa apenas **combinações aleatórias** em todos os espaços com nº de vezes fixos (n_iter)

- **Vantagem:** Viável para projetos grandes, funciona bem
- **Desvantagem:** Dependência de sorte, muita variância

Exemplo

```
# hiperparâmetros do KNN, agora como distribuições (não lista fixa)
param_dist = {
    'n_neighbors': randint(1, 30),          # sorteia inteiros entre 1 e 30
    'weights': ['uniform', 'distance']
}

random_search = RandomizedSearchCV(
    KNeighborsClassifier(), param_dist,
    n_iter=10,          # testa só 10 combinações aleatórias (não todas)
    cv=5, scoring='accuracy'
)

random_search.fit(X_train, y_train)
print(random_search.best_params_)
```

Como aplicar

Otimização Bayesiana

Usa os **resultados das tentativas anteriores** para construir um **modelo probabilístico** de quais regiões do espaço de hiperparâmetros parecem promissoras, e **escolhe o próximo ponto de forma inteligente**

- **Vantagem:** Eficiente e aprende com tentativas anteriores
- **Desvantagem:** Complexo, sequencial (causa lentidão), pode enviesar


```
import optuna
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import cross_val_score

def objective(trial):
    # espaço de busca: intervalo e categorias que o Optuna vai escolher
    n_neighbors = trial.suggest_int('n_neighbors', 1, 30)
    weights = trial.suggest_categorical('weights', ['uniform', 'distance'])

    # treina o modelo com a combinação sugerida nesta tentativa
    model = KNeighborsClassifier(n_neighbors=n_neighbors, weights=weights)

    # avalia com validação cruzada (5 folds) e tira a média
    score = cross_val_score(model, X_train, y_train, cv=5, scoring='accuracy').mean()
    return score # métrica que o Optuna tenta maximizar

# cria o estudo: direction='maximize' porque queremos a maior acurácia possível
study = optuna.create_study(direction='maximize')

# roda 30 tentativas, cada uma escolhida com base nos resultados anteriores
study.optimize(objective, n_trials=30)

# melhor combinação de hiperparâmetros encontrada
print(study.best_params)
```

Como aplicar

Algoritmos Genéticos

inspirada em **seleção natural/evolução biológica** ele trabalha com uma **população** de combinações de hiperparâmetros que evolui ao longo de gerações:

- **Vantagem:** Bom em espaços de busca grandes e complexos, com hiperparâmetros imprevisíveis
- **Desvantagem:** Muito Complexo, precisa de uma população de tentativas em cada geração, então também pode ser caro computacionalmente

```
import optuna
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import cross_val_score

def objective(trial):
    # espaço de busca: intervalo e categorias que o Optuna vai escolher
    n_neighbors = trial.suggest_int('n_neighbors', 1, 30)
    weights = trial.suggest_categorical('weights', ['uniform', 'distance'])

    # treina o modelo com a combinação sugerida nesta tentativa
    model = KNeighborsClassifier(n_neighbors=n_neighbors, weights=weights)

    # avalia com validação cruzada (5 folds) e tira a média
    score = cross_val_score(model, X_train, y_train, cv=5, scoring='accuracy').mean()
    return score # métrica que o Optuna tenta maximizar

# cria o estudo: direction='maximize' porque queremos a maior acurácia possível
study = optuna.create_study(direction='maximize')

# roda 30 tentativas, cada uma escolhida com base nos resultados anteriores
study.optimize(objective, n_trials=30)

# melhor combinação de hiperparâmetros encontrada
print(study.best_params)
```

Prática

Você deve ter notado que a uma biblioteca nova sendo usada em alguns exemplos

Optuna, uma biblioteca de otimização de hiperparâmetros, com foco em fazer isso de forma inteligente, com sintaxe simples e integração fácil com qualquer modelo

- **Define-by-run**: O espaço de busca é definido dentro da própria função objective, permite lógica condicional (ex: só testar max_depth se o modelo escolhido for uma árvore).
- **Múltiplos algoritmos**: Usa TPE (Otimização Bayesiana) por padrão, mas também suporta CMA-ES (Algoritmo Genético), Random e Grid.
- **Pruning**: Aborta tentativas ruins no meio do treino.

Guia para programar em optuna

1. A função `objective`

Sempre recebe `trial` como parâmetro, e sempre **retorna um número** (a métrica que você quer otimizar).

python

```
def objective(trial):  
    ...  
    return score
```

2. Os métodos - `trial.suggest_*`

como você define o espaço de busca

<code>trial.suggest_int('nome', min, max)</code>	# inteiro (ex: n_neighbors, max_depth)
<code>trial.suggest_float('nome', min, max)</code>	# decimal (ex: learning_rate, alpha)
<code>trial.suggest_float('nome', min, max, log=True)</code>	# decimal em escala log (bom pra learning_rate)
<code>trial.suggest_categorical('nome', ['a', 'b'])</code>	# escolhe entre opções fixas (ex: weights)

Guia para progamar em optuna

3. O cérebro - study

O objeto que gerencia toda a busca guarda o histórico de tentativas e decide com base nelas. Pode ser dividido em:

- **Maximize:** quando a métrica é "quanto maior melhor" (accuracy, F1, R^2)
- **Minimize:** quando é "quanto menor melhor" (erro, loss, RMSE)

```
python
```

```
study = optuna.create_study(direction='maximize') # ou 'minimize'
```

Guia para progamar em optuna

4. Rodar o código

Executa a função `objective` repetidamente, uma vez por tentativa, ajustando as escolhas a cada rodada.

```
study.optimize(objective, n_trials=30)
```

5. Resultados

```
study.best_params    # dicionário com a melhor combinação
study.best_value      # o valor da métrica nessa combinação
study.best_trial      # o objeto trial completo (com mais detalhes)
```

O código irá se parecer com isso:

```
# 1. função objective: recebe trial, retorna a métrica
def objective(trial):
    # 2. suggest_*: define o espaço de busca
    n_neighbors = trial.suggest_int('n_neighbors', 1, 30)
    weights = trial.suggest_categorical('weights', ['uniform', 'distance'])

    # cria e treina/avalia o modelo com os valores sugeridos
    model = KNeighborsClassifier(n_neighbors=n_neighbors, weights=weights)
    score = cross_val_score(model, X_train, y_train, cv=5, scoring='accuracy').mean()
    return score

# 3. cria o study: gerencia todo o processo de busca
study = optuna.create_study(direction='maximize')

# 4. roda a otimização: chama objective() n_trials vezes
study.optimize(objective, n_trials=30)

# 5. resultados finais
print(study.best_params) # melhor combinação de hiperparâmetros
print(study.best_value) # melhor valor da métrica alcançado
```


Conclusão

Os hiperparâmetros são usados para diversas finalidades, como:

1. Velocidade de treino/inferência

Às vezes você ajusta hiperparâmetros pra ganhar tempo, mesmo abrindo mão de um pouco de performance.

2. Dados desbalanceados

Hiperparâmetros não necessariamente tem a ver com complexidade do modelo, podem fazer o modelo prestar mais atenção na classe minoritária, quando o dataset tem muito mais exemplos de uma classe que de outra.

3. Velocidade de convergência

afeta quantas iterações (n_estimators) você vai precisar até o modelo estabilizar.

4. Uso de memória/recursos computacionais

Hiperparâmetros como max_depth ou n_estimators também controlam quanto de RAM/processamento o modelo consome

Agora responda o notebook e aplique seus conhecimentos

[Notebook](#)